

CHAPTER 1

INTRODUCTION

Underground water pipelines are a critical component of modern urban infrastructure, carrying potable water, waste, and industrial fluids across cities and towns. Over time, these pipelines experience wear and tear, leaks, blockages, and other issues that can lead to water wastage, contamination, and costly repairs. Traditional methods of detecting water flow or blockages in underground pipes often involve invasive procedures that can disrupt local infrastructure, leading to costly and time-consuming repairs.

In recent years, technological advancements have paved the way for innovative, non-invasive solutions to monitor underground water channels and detect pipeline faults in real-time. This project leverages these technological advancements to create an Arduino-based robotic system that can detect water flow in underground pipes using a Doppler radar sensor. The robot is designed to move within confined spaces, controlled via a Bluetooth-enabled mobile application, and collect data on water flow, which is crucial for identifying potential leaks or blockages.

The Bluetooth control system not only allows for the remote operation of the robot, but also enables real-time data transmission to the mobile app for monitoring and analysis. When water flow is detected, an auditory alert is triggered via a buzzer, allowing the operator to quickly identify sections of the pipeline that may require repair. This approach offers a non-invasive, efficient, and cost-effective method for managing underground water pipeline maintenance, improving both the detection of issues and the speed with which they can be resolved.

The goal of this project is to enhance the efficiency of water channel detection and to provide utility companies with a robust tool for timely and accurate maintenance of underground water infrastructure. By reducing the time and labor involved in detecting water leaks or flow interruptions, this system can contribute to more sustainable water management practices and minimize costly disruptions in water supply.

1.1 SCOPE OF WORK AND ITS IMPORTANCE

Traditional underground water pipe inspection and maintenance methods are inefficient, invasive, and costly. Excavating pipelines to detect leaks or blockages can cause significant disruption, leading to time delays and expensive repairs. The need for a real-time, non-invasive,

and remote-controlled solution to monitor underground water channels is paramount, particularly in urban areas where rapid intervention is crucial.

The objective of this project is to develop an Arduino-based robotic system equipped with a Doppler radar sensor to detect water flow within underground water pipes. The system should allow for real-time detection and alerting via a Bluetooth-controlled mobile app while also enabling remote control of the robot's movements. This solution aims to minimize the time and labor involved in detecting faults in water pipelines, improve the accuracy of fault detection, and reduce the overall cost and disruption caused by traditional methods. The system will alert the operator through a buzzer when water flow is detected, providing an immediate, efficient method for water pipe inspection and maintenance.

1.1.1 EXISTING SYSTEM

The current methodology for inspecting and maintaining underground water pipelines predominantly relies on external, indirect, and often disruptive techniques. These include acoustic leak detection, where ground microphones are used to listen for the sound of escaping water, and ground-penetrating radar (GPR), which is used from the surface to locate pipes and potential voids. For direct internal inspection, complex and costly systems like CCTV cameras on crawler robots are sometimes employed, but these are primarily for visual assessment of cracks or blockages and are not designed for active water flow analysis. A significant limitation of these methods is their inability to provide real-time, quantitative data on water flow velocity from within the pipe during operation. Furthermore, many require extensive excavation to access the pipeline, leading to high costs, prolonged downtime, and significant disruption to urban infrastructure and daily life.

1.1.2 PROPOSED SYSTEM

The proposed system introduces an innovative, Arduino-based robotic platform designed for direct, internal inspection of water pipes. This robot is equipped with a Doppler radar sensor to accurately detect and measure the velocity of water flow in real-time from inside the pipeline. The system is controlled wirelessly via a custom mobile application using a Bluetooth interface, allowing an operator to remotely navigate the robot through the pipes. All sensor data, including water flow information, is transmitted back to the mobile app for live monitoring and analysis. An additional feature includes an audible buzzer that activates immediately upon the detection of significant water flow, providing an instant alert to the operator. By utilizing DC motors for mobility, the robot offers precise movement control, enabling a comprehensive and efficient inspection process. This integrated solution aims to provide a non-destructive, real-time, and cost-effective method for underground water pipe assessment, thereby minimizing operational disruption and enhancing the precision of water management operations.

1.2 IMPORTANCE OF WORK

A ground-level water identification robot using radar and the Doppler effect has a wide range of practical applications across environmental, agricultural, and urban sectors. It can be used for flood monitoring by detecting rising water levels in roads, fields, and low-lying regions, helping authorities take early action. The robot is also useful in identifying underground water sources and moisture levels, which supports agriculture and borewell planning. In cities, it helps monitor drainage systems, detect waterlogging, and identify leakage in underground pipelines. Construction sites benefit from this technology as it can locate hidden water pockets beneath the soil, improving safety during foundation work. During natural disasters, the robot can safely navigate flooded areas to assess water depth and conditions, reducing human risk. Additionally, it supports environmental research by measuring surface water distribution and tracking groundwater recharge. Overall, this robot provides accurate, non-contact water detection, making it valuable for smart city management, disaster response, and scientific analysis.

- 1. Flood Detection and Early Warning:** The robot can be used to monitor ground-level water accumulation and rising water levels in flood-prone areas. By using radar signals and Doppler analysis, it identifies the movement and depth of water in real time. This helps authorities and communities receive early warnings during heavy rainfall, allowing quick evacuation and preventive measures. The robot's ability to work in dangerous or inaccessible areas makes it extremely useful for flood management.
- 2. Underground Water Source Identification:** The radar system in the robot can detect moisture levels and hidden water sources beneath the ground surface. This is highly beneficial for agricultural planning, irrigation design, and borewell drilling. Farmers and engineers can use the robot to understand where water is available, helping them make better decisions for water usage and land preparation.
- 3. Pipeline Leakage Detection:** Underground water pipelines often develop leaks that are difficult to detect manually. The robot can identify unusual water movement or moisture patterns using Doppler-based radar sensing. This helps detect leakage points early, reducing water wastage and preventing damage to infrastructure.
- 4. Urban Drainage and Sewage Monitoring:** In cities, drainage systems easily get clogged during rainy seasons. The robot can navigate near drainage lines and detect water levels, flow speed, and blockages. It provides continuous monitoring to prevent unexpected sewage overflow and waterlogging on roads.
- 5. Environmental and Research Applications:** Researchers can use the robot to study surface water distribution, groundwater recharge patterns, and environmental changes. The radar-based detection provides accurate data for climate studies, hydrological research, and ecosystem analysis. Its ability to operate in different terrains makes it a valuable tool for scientific monitoring.

1.3 TOOLS

The Ground-Level Water Identification Robot using Radar Based on the Doppler Effect it involves a wide range of tools and technologies that enables devices to connect, communicate and exchange data over the internet. Here are some key tools used in:

Radar Sensor: Detects water by sending radar waves and measuring the Doppler effect.

It provides accurate water presence and movement detection.

Microcontroller (Arduino/UNO): Used to detect water flow in underground pipes by measuring changes in frequency caused by moving water particles.

Bluetooth Module (HC-05 or HC-06): Enables wireless communication between the robot and the mobile app, allowing remote control and data transmission.

DC Motors and Motor Driver (L298N or similar): Provides movement capabilities for the robot, allowing it to navigate through confined spaces or over pipelines.

Chassis or Robotic Platform: A sturdy base that holds the Arduino, motors, and sensors in place, allowing the robot to move smoothly.

Wheels and Mounting Components: Attach to the motor shafts for smooth navigation on various surfaces.

Buzzer: An auditory alert that sounds when water flow is detected, notifying the operator in real-time.

Power Supply (Battery Pack): Provides necessary power for the robot and its components (motors, Arduino, sensors, and Bluetooth module).

Wires and Connectors: Essential for connecting all components together for smooth communication and data transfer.

Mobile Device (with Bluetooth capability): Required for running the mobile app, controlling the robot, and receiving data from the system.

1.3.1 Software Requirements:

Arduino IDE Used to write, compile, and upload code to the Arduino microcontroller. The software must include libraries to handle motor control, Bluetooth communication, and Doppler radar data processing.

1.3.2 Bluetooth Serial Terminal App:

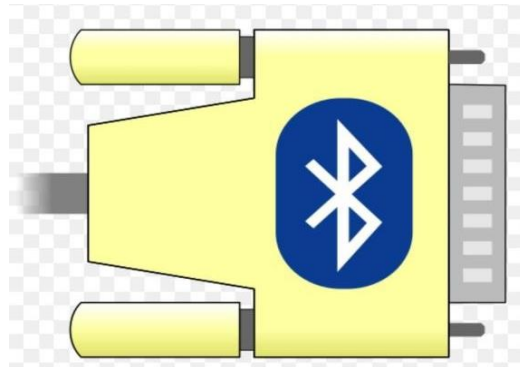


Fig 1.3: Bluetooth Serial Terminal App

The Bluetooth Serial Terminal app is a handy tool for wireless communication with Bluetooth-enabled devices. It connects to modules like HC-05 or HC-06 using the Serial Port Profile (SPP). This app is widely used by developers working on embedded systems and IoT projects. Users can send and receive serial data in real-time from their smartphones. It supports both ASCII and HEX formats for flexible communication. The app also offers features like command history, auto-reconnect, and line-ending settings. It's an essential utility for testing, debugging, and monitoring Bluetooth serial communication.

Procedure:

1. Install the "Serial Bluetooth Terminal" app from the Google Play Store.
2. Power on your robot and ensure the Bluetooth module, such as HC-05 or HC-06, is properly connected to the microcontroller.
3. Pair your mobile phone with the Bluetooth module using your phone's Bluetooth settings. The default PIN is usually 1234 or 0000.
4. Open the Bluetooth Serial Terminal app, go to the devices section, and connect to your Bluetooth module.
5. Once connected, use the terminal screen to send single-character commands to control the robot.

M1 to move forward

M2 to move backward

M3 to turn left

M4 to turn right

M5 to stop

M6 radar turn on

M7 radar turn off

6. For easier control, go to the “Keyboard” tab in the app and create custom buttons for each command so you can control the robot with one tap.

7. If a radar sensor is connected and programmed to send messages through Bluetooth, the app will display feedback like “Movement Detected” in the terminal when the sensor is triggered

CHAPTER 2

LITERATURE SURVEY

Abdullah Kadri; Adnan Abu-Dayya; Riccardo Stefanelli; Daniele Trincherro [1], The leak problem in water distribution systems has attracted the attention of many researchers during the past few decades. Water losses through leakages represent an issue that has negative impacts on the environmental and economic systems. Detecting and locating the source of leakage in the water distribution network can significantly reduce the amount of water lost and prevent the destruction of the infrastructure due to water leakage. In this paper, several methods for water leak detection are presented while indicating their advantages and drawbacks. In addition, a new technique based on using multiple sensing wires with different lengths is proposed. This solution consists in detecting and locating water leak by measuring the variations in soil moisture. The experiments made are presented to prove the simplicity, ease of use and the ability of the proposed technique to detect and pinpoint the location of water leak in underground pipelines.

Abdullah Kadri; Adnan Abu-Dayya; Riccardo Stefanelli; Daniele Trincherro [2], In this paper, characterization of an acoustic wireless sensor for leakage detection in underground pipes is presented. The sensor is based on an acoustic sensing element, called a hydrophone, which senses the noise inside the pipe. The sensor sends the sensed signals wirelessly to a gateway which is located on the ground. The gateway, in turn, relays these signals to a backend server for further processing. For concept-proofing, a novel experimental test-bed has been designed and realized in a water supply facility in Turin, Italy. In this paper, this test-bed is used to characterize the capability of the hydrophone of water leakage detection under high water pressure and under various leakage locations and volumes. The test-bed can also be used to test and evaluate the performance of any device that would operate in high pressure water pipes. The results show that the acoustic sensor is able to detect leakages with different magnitudes and at various locations.

Jian Xu; Mengqiang Duan; Xuge Li; Yuan Gao [3], The underground municipal water pipeline gallery project is a water conservancy engineering, with high risk, which makes important contributions to urban industrial and civil water use. Based on the 4M principle (Man/ Machine/ Medium/ Management) of accident prevention, this paper uses the systematic risk management theory method to identify and classify the risks of underground water pipeline gallery engineering from “Point-Line-Area-Volume”. The results show that the risk factors in the tunneling area are accounted for the largest proportion. The highest

risk factors appear in job positions. Based on the Internet of Things technology, cloud computing technology and wireless transmission technology, this paper proposes a new smart water conservancy system to monitor the risk factors. It is described for illustrating the architecture and functional modules. In addition, it is investigated about the application of these smart monitoring methods in actual projects. The results imply that the efficiency of intelligent safety monitoring methods is more than 7 times that of personnel inspections. The research conclusions can provide references for the implementation of risk identification and smart monitoring of underground municipal water pipeline gallery projects, and further improve the level of project safety management and disaster control.

DZIRI Jalal; Tahar Ezzedine [4], This paper represents the initial steps in the development of a new system for leak detection and localization in the water distribution network. This system is based on wireless underground sensor networks. First, we start with determining an optimal location of leak control points in the water distribution network especially in areas which are difficult to control. Then, we move to the development of a distributed algorithm to detect and locate in real time the small and the large leaks in the water distribution system. This algorithm is accompanied by a data aggregation method which minimizes the amount of transmitted information and avoids false alarms. In addition, we present a new cross-layer method in order to optimize the performance of the underground wireless sensor network.

Abdullah Kadri; Elias Yaacoub; Mohammed Mushtaha [5], In this paper, an empirical evaluation of acoustical signals for leakage detection and quantification in underground plastic pipes for main water distribution networks is presented. Several experiments have been carried out to collect acoustic signals generated from various leakage volumes. Upon signals analysis, it is noticed that the acquired signals are so weak and they are buried in the background noise caused by fluid flow, specially for small leakage volumes. In this work, a simple and easy to implement method for detecting and quantifying leakages in underground plastic pipes is developed. In this method, sound data are sampled and frequency spectrum of the acoustic signals is obtained by calculating the Fast Fourier Transform (FFT). Detection and quantification of existing leakages are carried out using the integral of the spectrum over frequencies below 500 Hz. The results obtained are very useful in cases where remote detection and quantification of existing leakages in underground plastic pipes are needed.

CHAPTER 3

BLOCK DIAGRAM

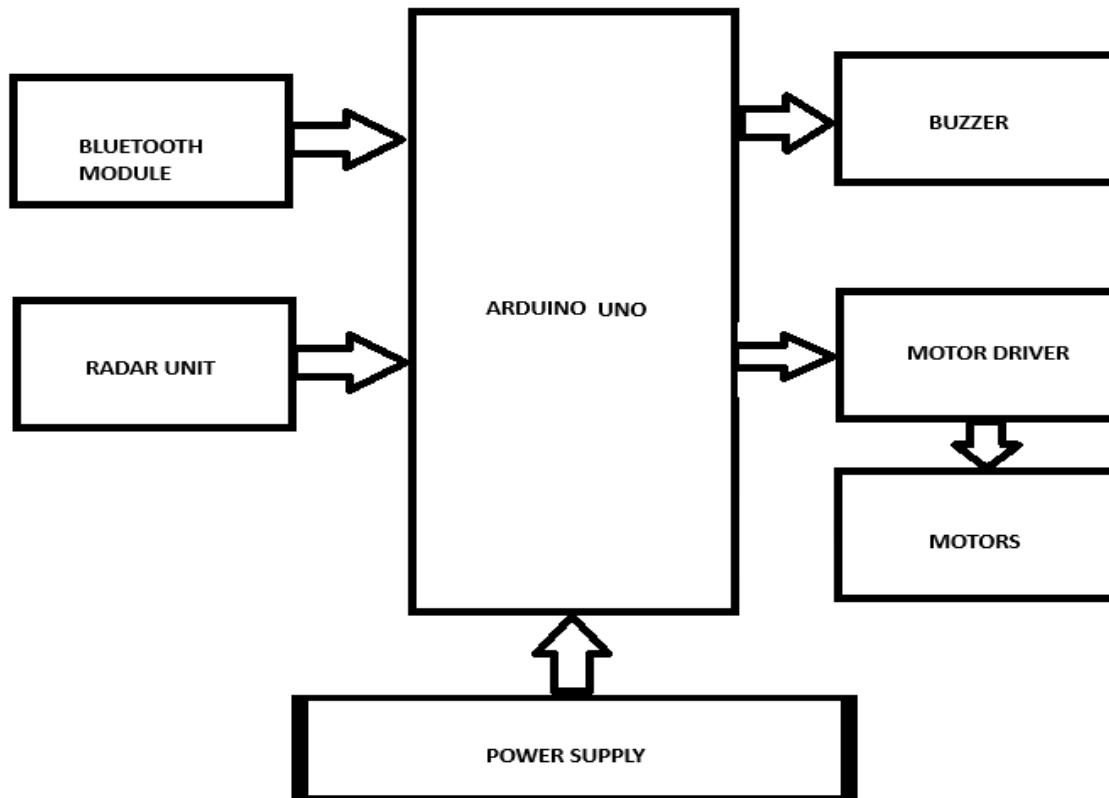


Fig 3.1: Block Diagram

3.1 ARDUINO UNO

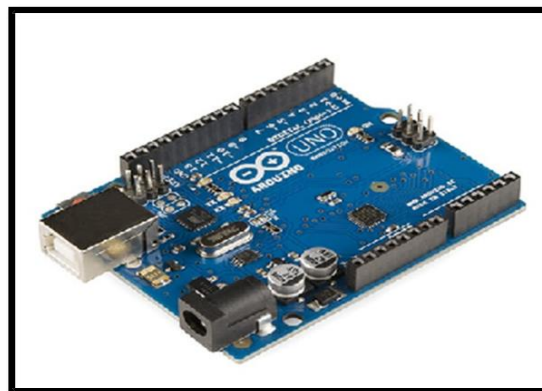


Fig 3.2 Arduino Uno Board

The Arduino Uno is an open-source microcontroller board based on the Microchip ATmega328P microcontroller (MCU) and developed by Arduino.cc and initially released in 2010. The microcontroller board is equipped with sets of digital and analog input/output (I/O) pins that may be interfaced to various expansion boards (shields) and other circuits. The board has 14 digital I/O pins (six capable of PWM output), 6 analog I/O pins, and is programmable with the Arduino IDE (Integrated Development Environment), via a type B USB cable. It can be powered by a USB cable or a barrel connector that accepts voltages between 7 and 20 volts, such as a rectangular 9-volt battery. It has the same microcontroller as the Arduino Nano board, and the same headers as the Leonardo board. The hardware reference design is distributed under a Creative Commons Attribution Share-Alike 2.5 license and is available on the Arduino website. Layout and production files for some versions of the hardware are also available.

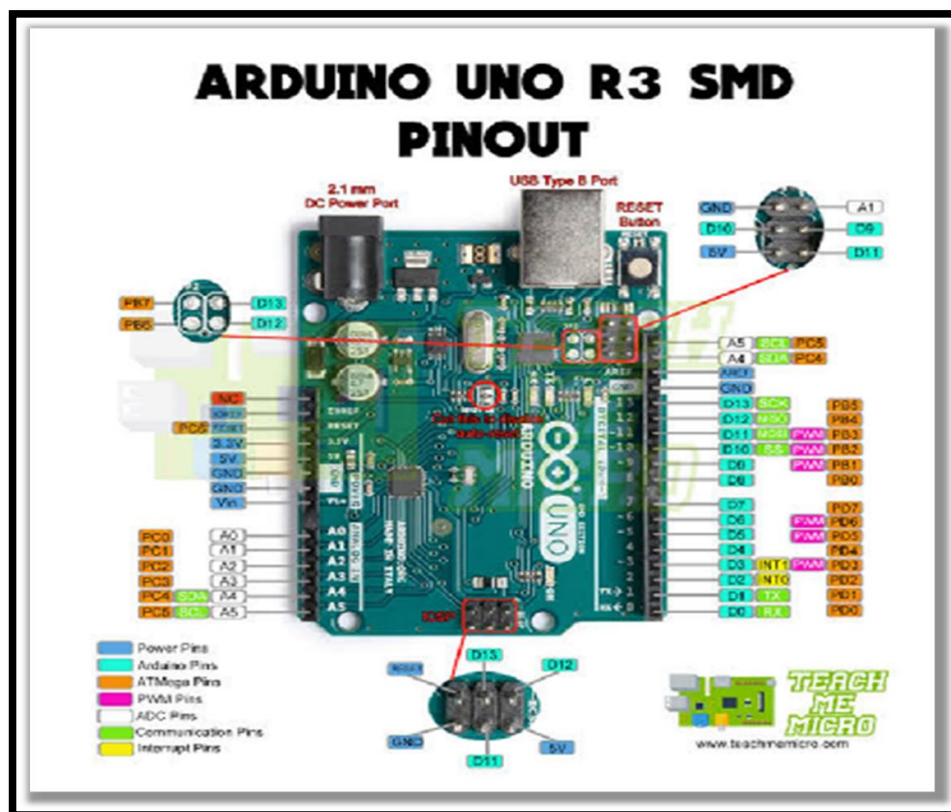


Fig 3.3: Arduino Pin Diagram

3.2 L298N 2A MOTOR DRIVER

L298N 2A Based Motor Driver is a high power motor driver perfect for driving DC Motors and Stepper Motors (Now find out out a wide range of motors at Robu.in).It uses the popular L298 motor driver IC and has an onboard 5V regulator which it can supply to an

external circuit. It can control up to 4 DC motors, or 2 DC motors with directional and speed control. This motor driver is perfect for robotics and mechatronics projects and perfect for controlling motors from microcontrollers, switches, relays, etc. Perfect for driving DC and Stepper motors for micro mouse, line following robots, robot arms, etc.

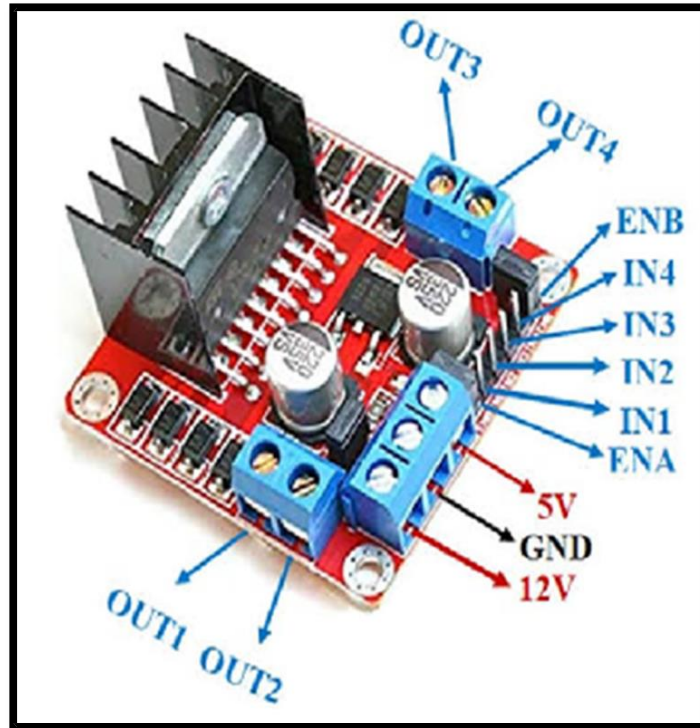


Fig 3.4: Motor Driver

3.3 DC BO MOTORS

The 300 RPM L-Shape BO Motor Plastic Gear Motor – BO series straight motor gives good torque and rpm at lower operating voltages, which is the biggest advantage of these motors. Small shaft with matching wheels gives an optimized design for your application



Fig 3.5: DC Motor

or robot. Mounting holes on the body & light weight makes it suitable for in-circuit placement. This motor can be used with 69mm Diameter Wheel for Plastic Gear Motors and 87mm Diameter Multipurpose Wheel for Plastic Gear Motors.

3.4 BLUETOOTH MODULE HC-06

HC-06 is the popular Bluetooth module. This HC06 module is slave mode only. It's very easy to add wireless serial connectivity for your device with this module. Examples for Arduino and other boards are available. Once you pair with other Bluetooth devices you work like with normal UART to exchange data. This module has built-in 3.3V voltage regulator and helps to break out the important pins (Vcc, Gnd, Txd, Rxd). Based on CSR BC4 chip, Bluetooth V2.0 + EDR. You can set the baud rate, name and pair password by AT commands when there is no Bluetooth connection. This module is a slave- it can be paired with Computer- Bluetooth master- mobile phone- PDA- PSP and so on.

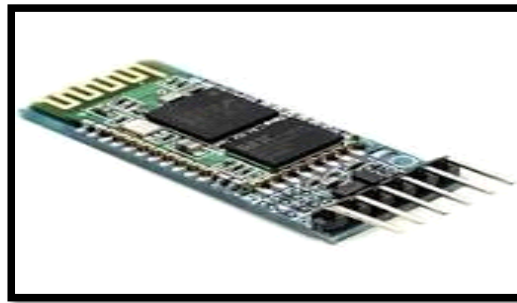


Fig 3.6: Bluetooth Module

3.5 MICROWAVE SENSOR(RADAR)



Fig 3.7: Microwave sensor

3.6 BATTERY

This LG INR18650 M26 2600mAh Lithium-Ion Battery gives value for your money. It comes with a rated voltage of 3.7 volts and a capacity of 2600mAh. It is a single cell, compact, and powerful battery cell with 2600 mAh capacity. It is very convenient to install in your project to fulfill the 3.7 Volt requirement with high capacity. The battery terminals can use in any compatible battery adapter/holder or it can be permanently soldered to your applications power source wires.



Fig 3.8: Battery

CHAPTER 4

SOFTWARE

4.1 Software Introduction:

Arduino IDE is an open-source software, designed by Arduino.cc and mainly used for writing, compiling & uploading code to almost all Arduino Modules. It is an official Arduino software, making code compilation too easy that even a common person with no prior technical knowledge can get their feet wet with the learning process. It is available for all operating systems i.e. (MAC, Windows, Linux and runs on the Java) Platform that comes with inbuilt functions and commands that play a vital role in debugging, editing and compiling the code. A range of Arduino modules available including Arduino Uno, Arduino Mega, Arduino Leonardo, Arduino Micro, and many more. Each of them contains a microcontroller on the board that is actually programmed and accepts the information in the form of code.

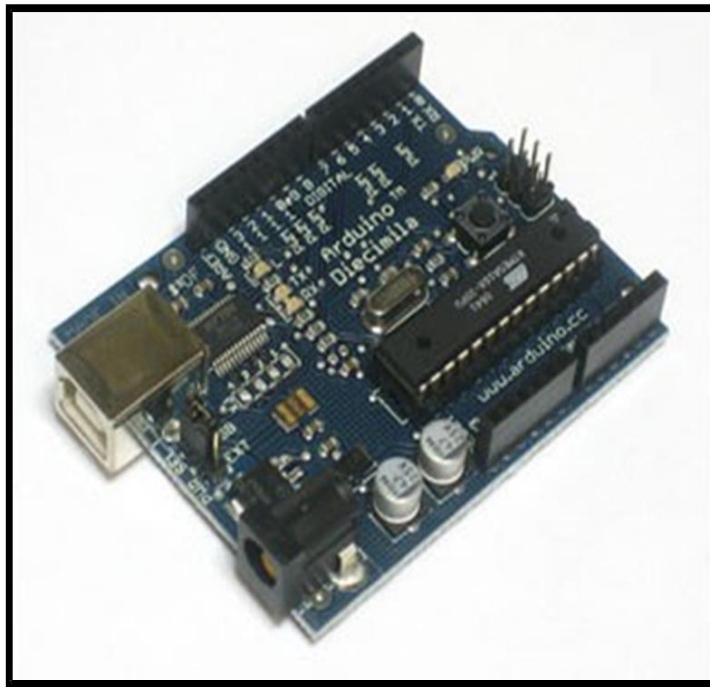


Fig 4.1: Arduino

4.1.1: Definition of Arduino IDE

Arduino is an open-source electronics platform based on easy-to-use hardware and software. Arduino boards are able to read inputs - light on a sensor, a finger on a button, or a Twitter message - and turn it into an output - activating a motor, turning

on an LED, publishing something online. You can tell your board what to do by platform. sending a set of instructions to the microcontroller on the board. To do so you use the Arduino programming language (based on Wiring) and the Arduino software (IDE), based on processing. Over the years, Arduino has been the brain of thousands of projects, from everyday objects to complex scientific instruments. A worldwide community of makers - students, hobbyists, artists, programmers, and professionals - has gathered around this open-source

4.1.2 About IDE

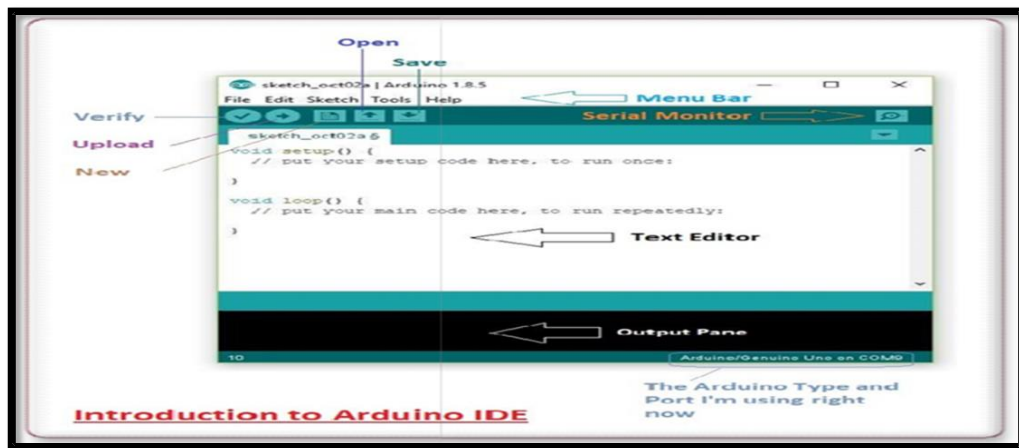


Fig 4.2: Menu Bar

Arduino ide is the software used to write-compile-upload program to Arduino. It's a open-source software, Procedure to Install Arduino Software (IDE)

Step 1 – First we must have an Arduino board and a USB cable.

In case we use Arduino UNO, Arduino Duemilanove, Nano, Arduino Mega 2560, or Diecimila, we will need a standard USB cable (A plug to B plug), the kind we would connect to a USB printer. In case we use Arduino Nano, we will need an A to Mini-B cable instead as shown in Fig. 4.3 (a).



Fig 4.3 (a) A to Mini-B Cable

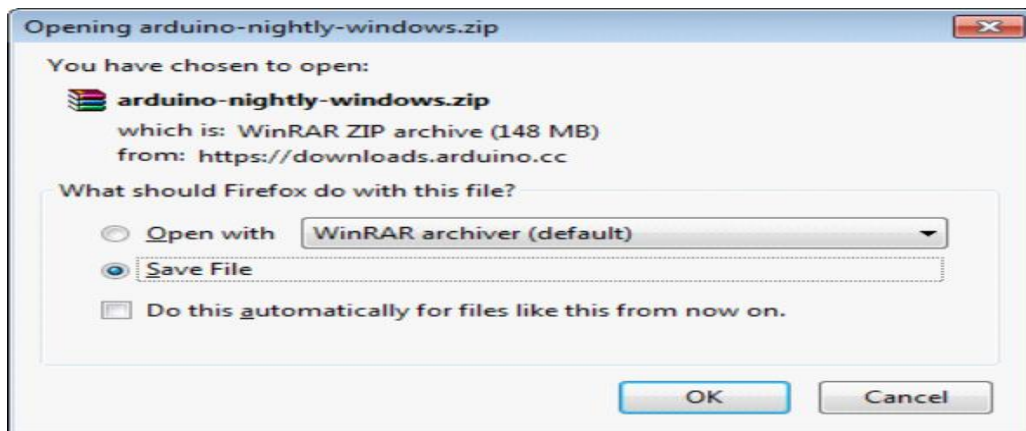
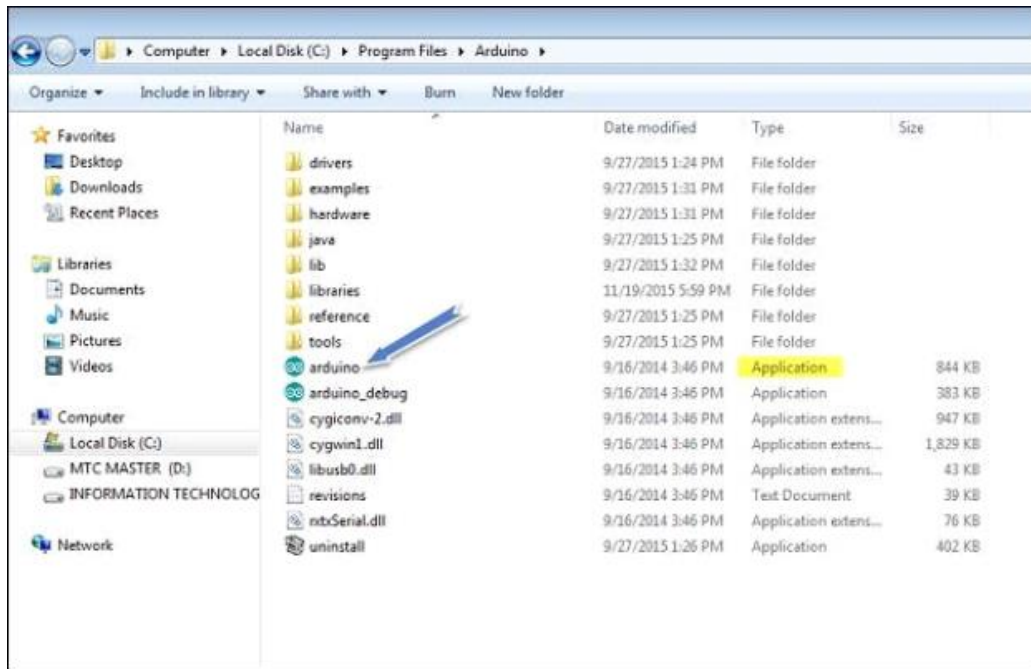
Step 2 – Download Arduino IDE Software.

Fig.4.3(b) Downloading Arduino IDE

Step 3 – Power up your board.

The Arduino Uno, Mega, Duemilanove and Arduino Nano automatically draw power from either, the USB connection to the computer or an external power supply. If you are using an Arduino Diecimila, you have to make sure that the board is configured to draw power from the USB connection. The power source is selected with a jumper, a small piece of plastic that fits onto two of the three pins between the USB and power jacks. Check that it is on the two pins closest to the USB port. Connect the Arduino board to computer using the USB cable. The green power LED (labeled PWR) should glow.

Step 4 – Launch Arduino IDE.**Fig. 4.4 Launching Arduino IDE**

your Arduino IDE software is downloaded, you need to unzip the folder. Inside the folder, you can find the application icon with an infinity label (application.exe). Double-click the icon to start the IDE.

Step 5 – Open your first project.

Once the software starts, we have two options:

- Create a new project.

To create a new project, select File → **New**, as shown in Fig. 3.1.5.5 (a).

- Open an existing project example.

To open an existing project example, select File → Example → Basics → Blink, as shown in Fig. 3.1.5.5 (b).

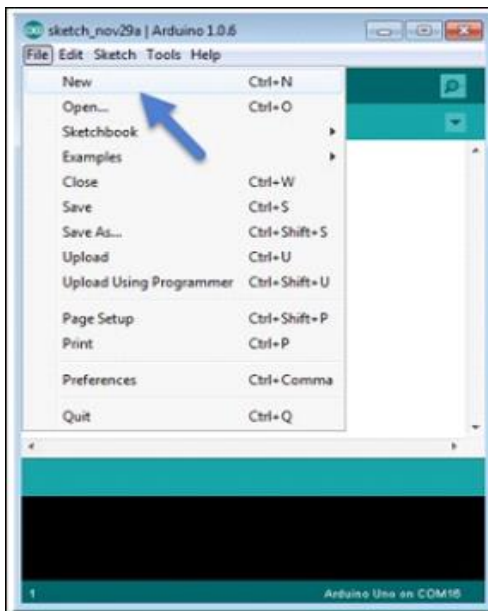
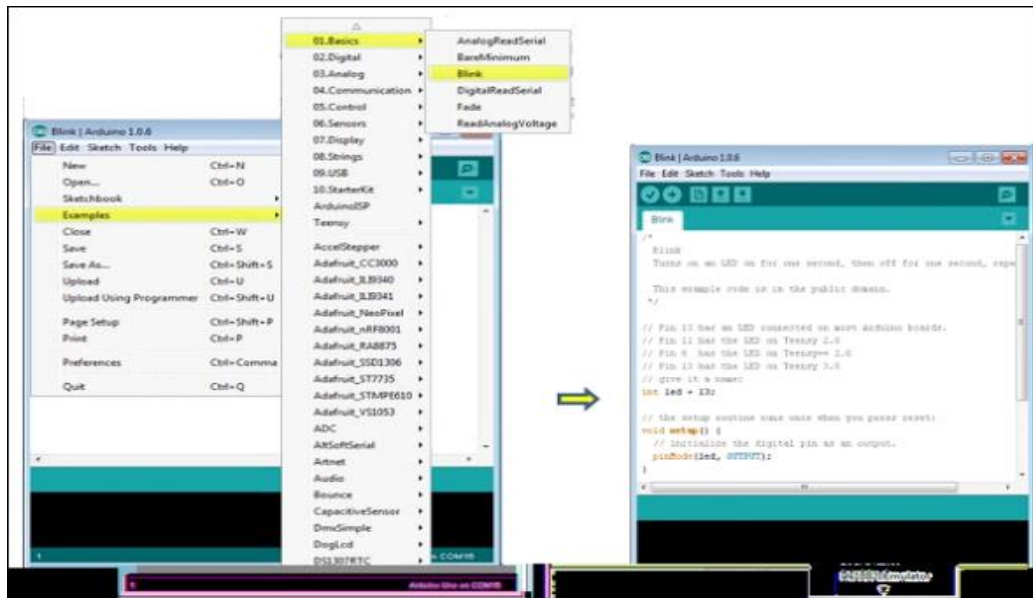


Fig. 4.5.1 (a) Creating a New Project



Fig. 4.5.2 (b) Opening an Existing Project

Here, we are selecting just one of the examples with the name Blink. It turns the LED on and off with some time delay. We can select any example from the list.

Step 6 – Select the respective Arduino board.

To avoid any error while uploading our program to the board, we must select the correct Arduino board name, which matches with the board connected to our computer. Go to Tools → Board and select the board.

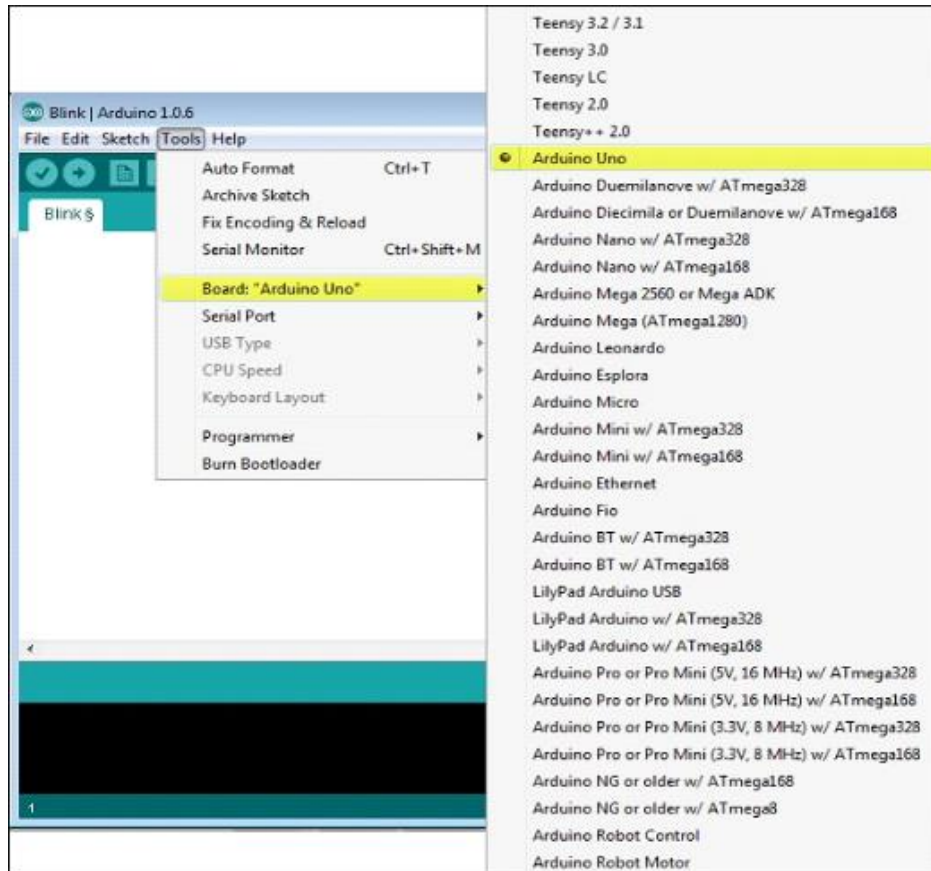


Fig. 4.6 Selecting the Arduino Board

Step 7 – Select the serial port.

Select the serial device of the Arduino board. Go to **Tools** → **Serial Port** menu.

This is likely to be COM3 or higher (COM1 and COM2 are usually reserved for hardware serial ports). To find out, you can disconnect your Arduino board and re-open the menu, the entry that disappears should be of the Arduino board. Reconnect the board and select that serial port.

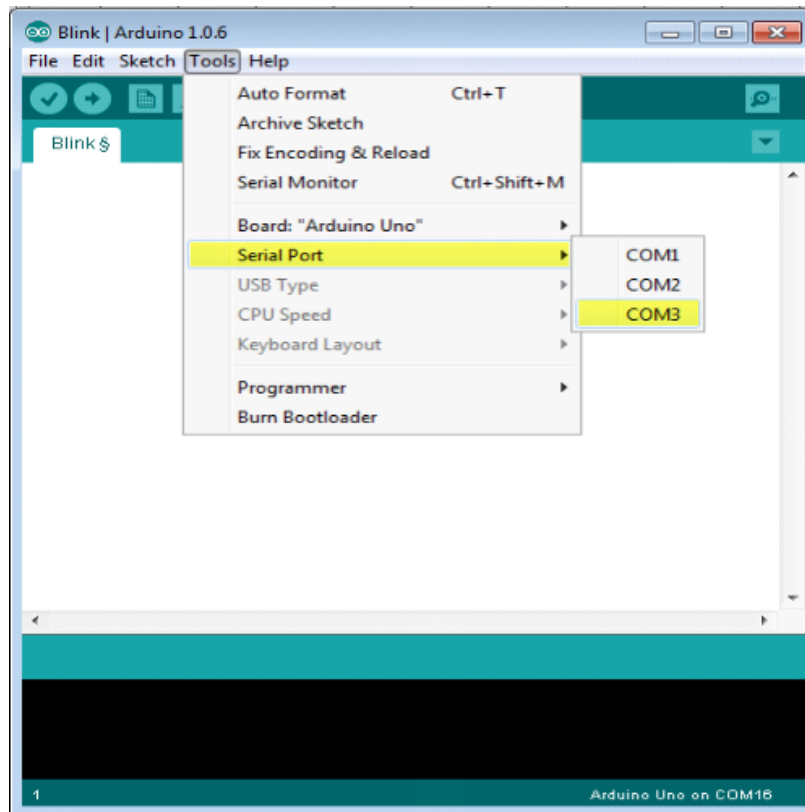


Fig. 4.7 Selecting the Serial Port

Step 8 – Upload the program to the board.

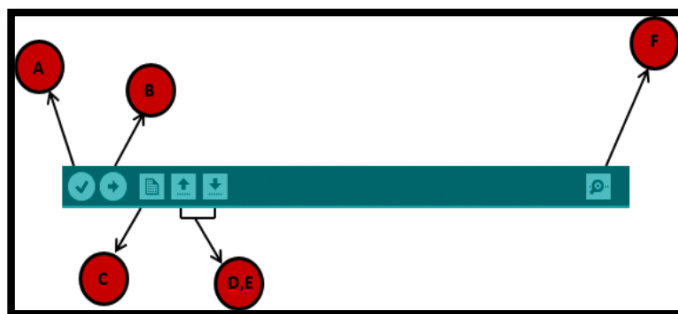


Fig. 4.8 Arduino IDE Toolbar

Before explaining how to upload our program to the board, we should know the function of each symbol appearing in the Arduino IDE toolbar.

A – Used to check if there is any compilation error.

B – Used to upload a program to the Arduino board.

C – Shortcut used to create a new sketch.

D – Used to directly open one of the example sketch.

E – Used to save your sketch.

F – Serial monitor used to send and receive the serial data from the board.

Now, simply click the "Upload" button in the environment. Wait few seconds; we will see the RX and TX LEDs on the board, flashing. If the upload is successful, the message "Done uploading" will appear in the status bar.

- **Program Structure**

In this section, we will study in depth, the Arduino program structure and we will learn more new terminologies used in the Arduino world. The Arduino software is open-source. The source code for the Java environment is released under the GPL and the C/C++ microcontroller libraries are under the LGPL. Arduino programs are called sketch.

Arduino programs can be divided in three main parts: **Structure**, **Values** (variables and constants), and **Functions**. Let us learn about the Arduino software program, step by step, and how to write the program without any syntax or compilation error. Let us start with the **Structure**. Software structure consist of two main functions:

- Setup() function
- Loop() function

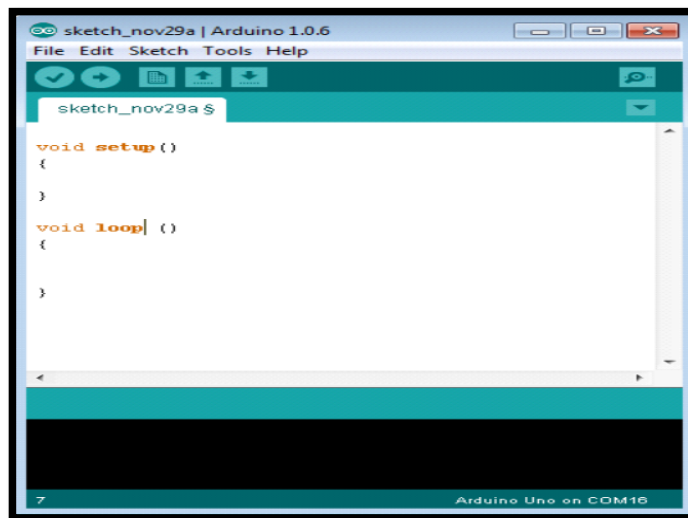


Fig. 4.9 Structure of a Sketch

The `setup()` function is called when a sketch starts. It is used to initialize the variables, pin modes, start using libraries, etc. The `setup` function will only run once, after each power up or reset of the Arduino board. After creating a `setup()` function, which initializes and sets the initial values, the `loop()` function does precisely what its name suggests, and loops consecutively, allowing our program to change and respond. It actively controls the Arduino board.

- **Programming using Embedded C**

C is a high-level programming language intended for system programming. Embedded C is an extension that provides support for developing efficient programs for embedded devices. Yet, it is not a part of the C language. In our Internship program, we employed Embedded C programs to write sketches to be dumped on Arduino Uno.

- **Introduction to Embedded C**

Embedded C programming language is an extension to the traditional C programming language that is used in embedded systems. The embedded C programming language uses the same syntax and semantics as the C programming language. The only extension in the Embedded C language from normal C Programming Language is the I/O Hardware Addressing, fixed-point arithmetic operations, accessing address spaces, etc.

- **Libraries**

In Arduino, much like other leading programming platforms, there are built-in libraries that provide basic functionality. In addition, it's possible to import other libraries and expand the Arduino board capabilities and features. These libraries are roughly divided into libraries that interact with a specific component or those that implement new functions.

To import a new library, we need to go to Sketch > Import Library

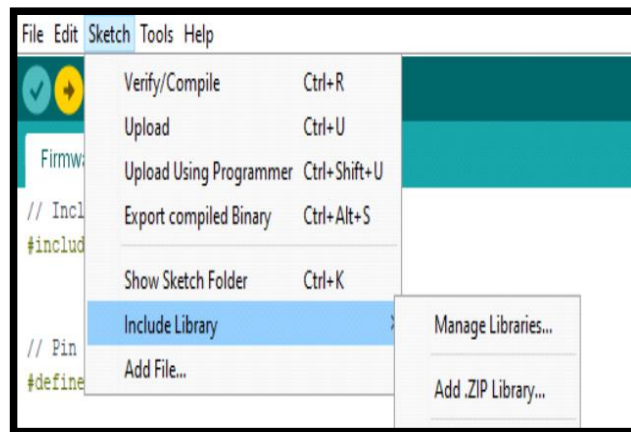


Fig. 4.10 Including Libraries on IDE

4.2 PROGRAM

```
const int sensorPin = 3;    // Sensor connected to pin 2

volatile int pulseCount = 0; // Variable to count pulses

unsigned long previousMillis = 0;

const unsigned long interval = 3000; // 3 seconds interval

char BluetoothData; // the Bluetooth data received

int cpmValues[5] = {0};    // Array to store the last 5 CPM values

int indexP = 0;            // indexP for storing CPM values

int sum = 0;               // Sum of the last 5 CPM values

int averageCPM = 0;        // Average of the last 5 CPM values

int left_motor_1 = 4; // left motor first pin to pin 4
```

```
int left_motor_2 =5; // left motor second pin to pin 5

int right_motor_1 =6; // right motor first pin to pin 7

int right_motor_2 =7; // right motor second pin to pin 8

int buz=13;

bool radar=false;

void setup() {

pinMode(sensorPin, INPUT);

attachInterrupt(digitalPinToInterrupt(sensorPin), countPulse, RISING); //
Interrupt on rising edge

Serial.begin(9600);

pinMode(buz,OUTPUT);

digitalWrite(buz,0);

pinMode(left_motor_1,OUTPUT);

pinMode(left_motor_2,OUTPUT);

pinMode(right_motor_1,OUTPUT);

pinMode(right_motor_2,OUTPUT);


//Set State of Pins all to LOW (0)

digitalWrite(left_motor_1,0);

digitalWrite(left_motor_2,0);

digitalWrite(right_motor_1,0);

digitalWrite(right_motor_2,0);

}

void loop() {
```

```
if (Serial.available()){

  BluetoothData=Serial.read(); //Get next character from Bluetooth

  if(BluetoothData=='1'){ // Red Button Pressed

    forward();

  }

  if(BluetoothData=='2'){ // Yellow Button Pressed

    backward();

  }

  if(BluetoothData=='5'){ // Red or Yellow Button Released

    stopcar();

  }

  if(BluetoothData=='3'){ // Green Button Pressed

    left();

  }

  if(BluetoothData=='4'){ // Blue Button Pressed

    right();

  }

  if(BluetoothData=='6'){ // Blue Button Pressed

    radar=true;

    pulseCount=0;

    Serial.print("Radar On");

  }

  if(BluetoothData=='7'){ // Blue Button Pressed

    radar=false;

    digitalWrite(buz,0);

    Serial.print("Radar Off");
```

```
}  
  
}  
  
if(radar){  
  
    unsigned long currentMillis = millis();  
  
  
    // Check if 3 seconds have passed  
  
    if (currentMillis - previousMillis >= interval) {  
  
        previousMillis = currentMillis;  
  
  
        // Calculate counts per minute (CPM)  
  
        int cpm = pulseCount * 20; // 20 * 3 = 60 seconds to calculate CPM  
  
        pulseCount = 0;          // Reset pulse count  
  
        // Store the current CPM value and update the average  
  
        cpmValues[indexP] = cpm;  
  
        indexP = (indexP + 1) % 5; // Move to the next indexP (0-4)  
  
        // Calculate the sum of the last 5 CPM values  
  
        sum = 0;  
  
        for (int i = 0; i < 5; i++) {  
  
            sum += cpmValues[i];  
  
        }  
  
  
  
        // Calculate the average CPM  
  
        averageCPM = sum / 5;  
  
        if(averageCPM > 50){  
  
            digitalWrite(buz,1);
```

```
Serial.print("Water detected, lat:");

} else {

digitalWrite(buz,0);

}

Serial.print(" Average CPM: ");

Serial.println(averageCPM);

}

}

}

// Interrupt service routine to count pulses

void countPulse() {

pulseCount++;

}

void forward(){

digitalWrite(left_motor_1,1);

digitalWrite(left_motor_2,0);

digitalWrite(right_motor_1,0);

digitalWrite(right_motor_2,1);

}

void backward(){

digitalWrite(left_motor_1,0);

digitalWrite(left_motor_2,1);
```

```
digital Write(right_motor_1,1);  
  
digitalWrite(right_motor_2,0);  
  
}
```

```
void left(){  
  
digitalWrite(left_motor_1,1); //Turn digital out left_motor_1 to High  
digitalWrite(left_motor_2,0); //Turn digital out left_motor_2 to Low  
digitalWrite(right_motor_1,1); //Turn digital out right_motor_1 to High  
digitalWrite(right_motor_2,0); // turn digital out right_motor_2 to low  
  
}
```

```
void right(){  
  
digitalWrite(left_motor_1,0);  
digitalWrite(left_motor_2,1);  
digitalWrite(right_motor_1,0);  
digitalWrite(right_motor_2,1);  
  
}
```

```
void stopcar(){  
  
digitalWrite(left_motor_1,0);  
digitalWrite(left_motor_2,0);  
digitalWrite(right_motor_1,0);  
digitalWrite(right_motor_2,0);  
  
}
```

CHAPTER 5

SUMMARY OF WORK

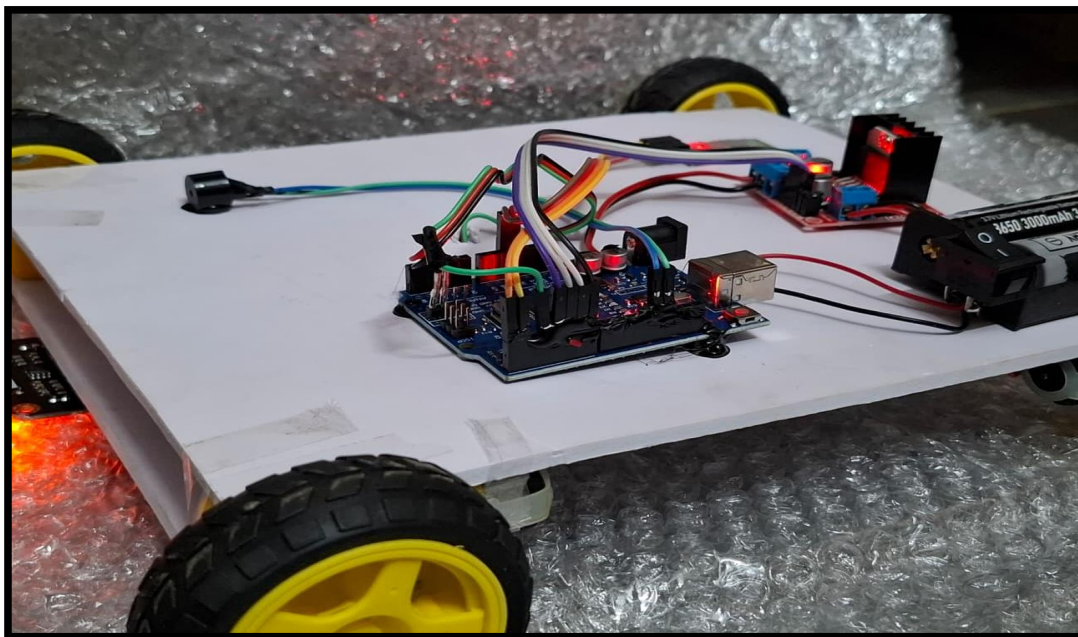


Fig5.1: Hardware implementation

The methodology of this project involves a combination of hardware components, software programming, and communication protocols to achieve real-time detection and reporting of water flow in underground channels. Arduino Microcontroller is the heart of the system, responsible for controlling the robot's movements and interfacing with the Doppler radar sensor, Bluetooth module, motors, and buzzer. Doppler Radar Sensor This sensor is used to detect the presence of water flow in underground pipes by measuring the changes in the frequency of waves that reflect off moving water particles. DC Motors and Motor Driver is the robot is equipped with motors that allow it to navigate through underground spaces or around pipe channels. These motors are controlled via a motor driver connected to the Arduino, with inputs from the Bluetooth controller. Bluetooth Module, This module allows for wireless communication between the robot and the mobile app, enabling remote control of the robot's movement and data transmission of the detected water flow to the user's mobile device. Buzzer, The buzzer is programmed to sound when water flow is detected by the Doppler radar, providing an auditory alert to the user. The system's software is developed using the Arduino IDE, with appropriate libraries for controlling the motors, Bluetooth module, and radar sensor. Bluetooth

Communication, A mobile application is designed to interface with the robot via Bluetooth, providing a user-friendly interface for controlling the robot's movement (forward, backward, left, and right). The app also displays real-time data on water flow as detected by the radar. Data Processing and Alerts is the Doppler radar sensor continuously sends data to the Arduino, which processes this information to detect any anomalies or water flow in the pipes. When water flow is detected, the system triggers the buzzer and transmits the data to the mobile app for display. Robot Movement and Control. The user can control the robot's movement using the mobile app, which sends commands via Bluetooth to the Arduino. The Arduino then sends signals to the motor driver, which moves the motors according to the user's inputs, allowing for full directional control of the robot. Data Transmission and Notification The Doppler radar's data is transmitted wirelessly to the mobile app via Bluetooth. The app processes the data in real-time, displaying whether water flow is present and the approximate strength of the flow. If water flow is detected, the app can log the data for further analysis. An additional feature includes triggering the buzzer when water flow is detected, providing an immediate auditory signal to the user in the field. Testing and Calibration, The system undergoes testing in various underground water pipe scenarios to ensure accurate detection of water flow. Calibration of the Doppler radar is done to fine-tune the sensitivity and accuracy of water flow measurements.

Fig5.2: Mobile device with Bluetooth serial terminal app

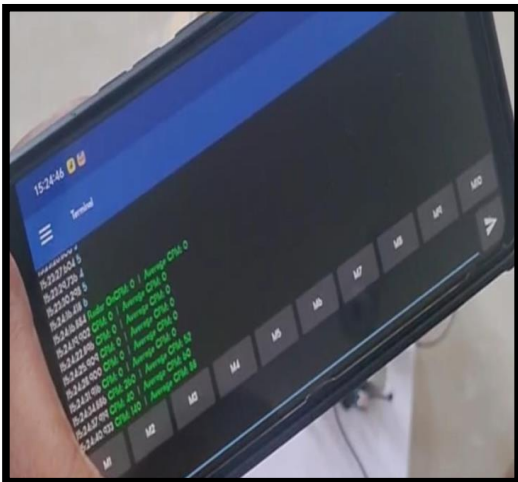
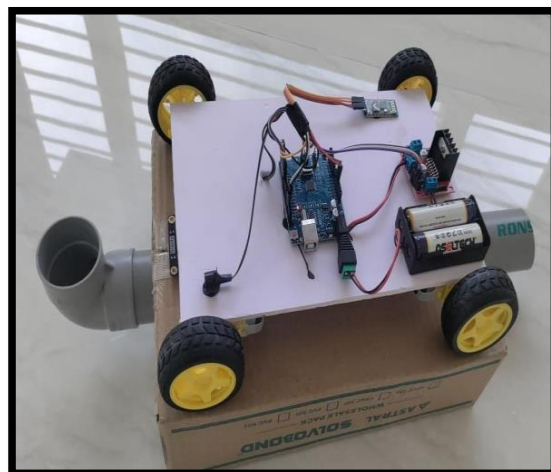


Fig 5.3 : Final Model



ADVANTAGES

1. **Non-Destructive Inspection:** Eliminates or minimizes the need for extensive and costly excavation to access pipes.
2. **Real-Time Data:** Provides immediate feedback on water flow conditions, enabling quick decision-making.
3. **Cost-Effective:** Built with affordable, off-the-shelf components (Arduino, DC motors), making it significantly cheaper than commercial inspection crawlers.
4. **Precision Location:** Can pinpoint the exact location of a problem (like a leak) from within the pipe, improving repair accuracy.
5. **Enhanced Safety:** Reduces the need for workers to enter confined or dangerous underground spaces.
6. **User-Friendly Operation:** Controlled via a simple mobile app interface, reducing the need for highly specialized training.
7. **Active Flow Measurement:** The Doppler radar provides quantitative data on water movement, which is superior to simple visual inspection.
8. **Rapid Deployment:** The small and portable system can be deployed quickly through standard manholes or access points.

CONCLUSION

In conclusion, this project successfully proposes the design and development of an innovative, Arduino-based robotic system for the internal inspection of underground water pipes. By leveraging Doppler radar technology, the robot offers a significant improvement over traditional external inspection methods by providing real-time, quantitative data on water flow from within the pipeline. The integration of Bluetooth for remote control and data transmission, coupled with an audible alert system, creates a user-friendly and efficient solution. While the prototype has limitations in range and operational time, it establishes a cost-effective and versatile platform for non-destructive pipe assessment. This system has the potential to revolutionize urban water management practices by enabling proactive maintenance, minimizing disruptive excavations, and ensuring the longevity and efficiency of critical water infrastructure.

REFERENCES

- [1][Arduino.cc](https://www.arduino.cc/en/Guide). (2023). *Arduino Documentation*. <https://www.arduino.cc/en/Guide>
- [2]RCWL-0516 Microwave Radar Sensor Datasheet.
- [3]L298N Motor Driver Module Datasheet.
- [4]Smith, J., & Doe, A. (2021). *A Review of In-Pipe Inspection Robots*. Journal of Robotics and Automation, 10(2), 45-60.
- [5]MIT App Inventor. (2023). *Create Apps for Android*. <http://appinventor.mit.edu/>
- [6]Kumar, R. (2020). *Microcontroller Based Robotics Projects*. Tech Publications.

